# AI Scholar

Intelligent Learning & Career Success Ecosystem

Unified Project Plan and Database Schema
— Reconciled, Implementation-Ready Edition —

A single source of truth merging the development roadmap and the PostgreSQL/Supabase schema, with every inconsistency between the two resolved so that each table, trigger, and policy maps directly onto a feature described in the plan.

| | |
|---|---|
| **Prepared for:** | Ragu |
| **Stack:** | Next.js, FastAPI, Supabase (PostgreSQL + pgvector), JWT/Supabase Auth |
| **Track:** | Placement Portfolio Project — AI/ML Development |
| **Date:** | June 28, 2026 |

This document supersedes the standalone project-plan and schema-design drafts.
See §6 for the API contract layer and Appendix A/B for the reconciliation changelog and field-parity tables.

# Contents

# 1 Core Insight Before You Build

AI Scholar as originally conceived is not one project — it is roughly six products bolted together: a RAG-based tutor, a quiz/flashcard generator, a study planner, a GitHub/profile analyzer, a coding-prep engine, and a career roadmap generator. Attempting to build all of it at once leads to months of work with nothing demoable.

**Strategy:** find the thinnest vertical slice that works end-to-end, deploy it, then layer modules on top. Since this project exists for personal development and placements, the real risk is not the deadline — it is that nothing forces a release. A small, deployed, live-linkable version beats a 60%-built ecosystem that never goes public.

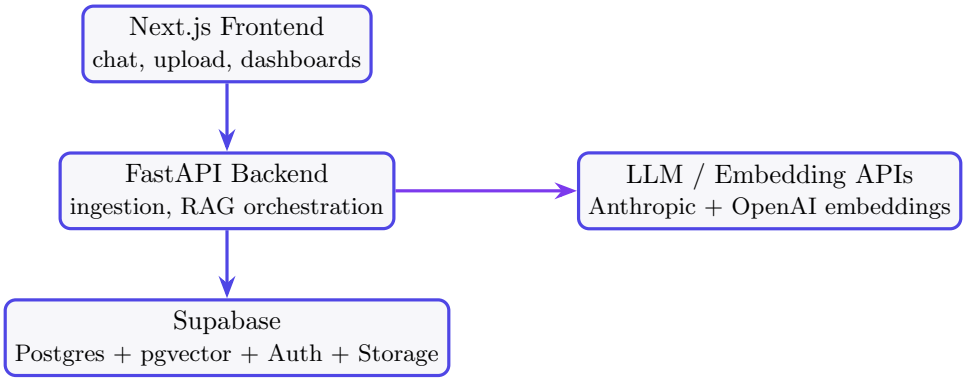
## 1.1 What Changes Because This Is for Placements

- **Depth beats breadth.** Interviewers probe the hard parts of a project instantly. One well-built module with real engineering trade-offs is a stronger story than six shallow ones glued from tutorials.
- **Each phase yields a resume bullet + an interview story.** Build where genuinely hard problems live (retrieval quality, evaluation, grounding) and go deep there.
- **Deploy early** — at the end of Phase 1, not Phase 4. A live URL a recruiter can click is worth more than extra modules on localhost.
- **Dogfood the career module on yourself.** Feed it your own GitHub, resume, and target companies. This surfaces what is actually useful and produces a memorable interview narrative.

# 2 System Architecture Overview

Almost every feature depends on one core capability: the system reading a student's material and answering questions grounded in it (RAG — Retrieval-Augmented Generation). Quizzes, flashcards, revision notes, and study plans all sit on top of that retrieval layer. It is also the highest technical risk component, so it is built first and proven before anything else.

**Scope discipline:** start with text-based documents only — PDFs, typed notes, slide decks, research papers. Handwritten notes (needs OCR) and lecture recordings (needs audio transcription) are deliberately deferred to Phase 5. They add large complexity for a fraction of early value.




## 2.1 Recommended Tech Stack

- **Frontend:** Next.js / React.

- **Backend:** FastAPI (Python) — best ecosystem for the AI/RAG side.
- **LLM orchestration:** Anthropic API for generation; LangChain/LlamaIndex used lightly if at all (they can become a black box).
- **Embedding model (finalized): OpenAI** `text-embedding-3-small`, dimension **1536**. The embedding model is independent of the generation model, so pairing OpenAI embeddings with Anthropic generation is standard practice. This decision is locked before Phase 1 begins — see §5.1.1 for why it cannot be deferred.
- **Vector database:** pgvector inside Supabase Postgres.
- **File storage:** Supabase Storage, private bucket.
- **Auth + DB:** Supabase (bundles auth, Postgres, and storage).

**Reuse opportunity:** the resume-parsing and career-analysis logic from the Signuture project overlaps directly with AI Scholar's career module (Phase 3). Reuse that ingestion logic instead of rebuilding it.

# 3 Phase Breakdown

Phase numbers below are the canonical numbering for the whole project. Every schema table in §5 is tagged with the phase that introduces it, so the plan and the schema always refer to the same phase for the same feature.

## 3.1 Phase 0 — Scoping & Setup (˜1 week)

Lock down one primary user story for the MVP: *"a student uploads a PDF and gets accurate, cited answers + a quiz."* Set up the repository, Supabase Auth, file upload, database schema, and a deployment target early.

**Schema introduced:** `profiles` table and its auto-creation trigger; `documents` table (structure only — the RAG pipeline that populates it is Phase 1); the private `documents` storage bucket. No quiz, study-plan, or career tables are created yet, even though their shape is decided now.

## 3.2 Phase 1 — RAG Tutor MVP (2–4 weeks)

Upload a PDF → extract text → chunk → embed → store → retrieve → answer with citations. The deliverable is a working chat-over-your-notes. Retrieval quality (chunk size, overlap, re-ranking) is the priority here, because everything downstream inherits it. Do not move on until answers are genuinely grounded and not hallucinated.

**Deploy at the end of this phase.** Push to Vercel/Railway/Render and get a real URL the moment the upload-and-chat loop works.

**Schema introduced:** `document_pages`, `document_chunks`, `conversations`, `messages`, `message_sources`, the `match_chunks` retrieval function, and the HNSW vector index.

## 3.3 Phase 2 — Learning Tools Layer (2–3 weeks)

Reuse the retrieved context for quiz generation, flashcards, revision-note summaries, and a basic study planner. These are mostly prompt engineering and UI on top of Phase 1, so they move fast. Add lightweight progress tracking (what has been studied, quiz scores).

**Schema introduced:** `quizzes`, `quiz_questions`, `quiz_attempts`, `quiz_answers`, `study_plans`, `study_plan_items`, `learning_progress`.

### 3.4 Phase 3 — Career & Profile Module (3–4 weeks)

Effectively a second product — treated as its own milestone. Pull GitHub repos (GitHub API), parse a resume (reuse Signuture), optionally read a LeetCode/Codeforces profile. Run skill-gap analysis against a target role/company and produce concrete gap analysis, framed carefully (see caution below).

**Caution:** readiness scores and roadmaps are where these tools usually feel generic. Make this specific — grounded in actual repositories and actual role requirements — or skip the single "score" number and present concrete gap analysis instead. **Decision point:** consider folding this module into Signuture given the heavy overlap in resume-ingestion logic, rather than maintaining two separate implementations.

**Schema introduced:** `career_profiles`, `user_skills`, `target_roles`, `gap_analyses`.

### 3.5 Phase 4 — Integration, Personalization & Polish (2–3 weeks)

Tie academic progress and career data into one profile, add a "digital mentor" thread that references a student's history, clean up the UI, handle errors and edge cases. This is the demo-ready / portfolio-ready milestone.

**Schema introduced:** no new tables. Phase 4 is application logic and UI over data that already exists. The one schema-level addition is the `student_overview` view (§5.6), which assembles the unified profile read by the dashboard and the mentor thread.

### 3.6 Phase 5 — Multimodal & Scale (Stretch / Post-MVP)

Add handwritten-note OCR, lecture-audio transcription (e.g. Whisper), and any scaling work. Real features, but they belong after a working core — not before.

**Schema impact:** none built now. Forward-compatibility notes are recorded in §5.11 so that Phase 5 does not require destructive migrations when it eventually starts.

### 3.7 Phase Cross-Reference

| Phase | Plan Deliverable | Schema Components |
|---|---|---|
| 0 | Repo, auth, upload skeleton | `profiles` + auto-create trigger, `documents` (structure) |
| 1 | RAG tutor MVP, deployed | `document_pages`, `document_chunks`, `conversations`, `messages`, `message_sources`, `match_chunks`, HNSW index |
| 2 | Quizzes, flashcards, study planner | `quizzes`, `quiz_questions`, `quiz_attempts`, `quiz_answers`, `study_plans`, `study_plan_items`, `learning_progress` |
| 3 | Career & profile module | `career_profiles`, `user_skills`, `target_roles`, `gap_analyses` |
| 4 | Integration & polish | `student_overview` view only |
| 5 | Multimodal (stretch) | none — forward-compatibility notes only |

## 4 Honest Cautions

- The career-readiness and skill-gap pieces are easiest to make impressive on a slide and hardest to make actually useful. Budget extra time there.

- Handwritten OCR and audio transcription sound compelling and tempt early starts. Resist — they are Phase 5 for a reason.
- If working solo, Phases 0–2 alone are a complete, modern, hireable portfolio piece. Phases 3+ can be framed as "and I'm extending it with X," signalling an active builder.
- Signuture and a research paper are already in progress. One deep, deployed project plus Signuture is a stronger placement story than three half-finished things.

### Suggested Minimum Viable Scope

Treat Phases 0–2 (RAG tutor + learning tools, deployed) as the real project. Everything else becomes ongoing, demonstrable extension work — which reads as momentum, not as an unfinished mess.

# 5 Database Schema

## 5.1 Design Principles

- **Supabase** provides Postgres, Auth, Storage, pgvector, and Row-Level Security in one managed platform.
- `auth.users` is managed by Supabase directly — no duplicate users table is created. A `profiles` table extends it.
- PDF files live in Supabase Storage; PostgreSQL stores only metadata.
- Every table holding student data has Row-Level Security enabled *and* an explicit policy — enabling RLS without a policy denies all non-service-role access by default, which silently breaks the app rather than protecting it.
- Every foreign key has an explicit `ON DELETE` behavior. Postgres defaults to `RESTRICT` when none is given, which would otherwise make a document permanently undeletable once it has been cited in a single chat answer or quiz question.
- `CHECK` constraints are used in place of native Postgres `ENUM` types for all status/role fields. This keeps adding a new valid value a simple `ALTER TABLE ... DROP CONSTRAINT / ADD CONSTRAINT`, rather than the more invasive `ALTER TYPE` migration enums require.
- All timestamp columns use `TIMESTAMPTZ`, not `TIMESTAMP`, so stored values are unambiguous across time zones.

### 5.1.1 Embedding Dimension — A Phase 0 Decision, Not a Deferred One

`pgvector`'s `vector(N)` type requires a fixed dimension *at column creation*. There is no "decide later" option: changing the dimension after documents have been embedded means dropping the column and re-embedding every chunk from scratch. The model is therefore finalized now: **OpenAI** `text-embedding-3-small`, `vector(1536)`. If a different model is substituted later, every `vector(1536)` column and the `match_chunks` function signature below must be updated together.

## 5.2 Phase 0 — Identity and Document Foundation

### 5.2.1 profiles

Central identity anchor for both the learning pipeline and the career module. A trigger on `auth.users` guarantees every signup gets a matching profile row — without it, new users would be orphaned and every downstream foreign key referencing `profiles(id)` would fail for them.

```
CREATE TABLE profiles (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    full_name TEXT,
    email TEXT,
    avatar_url TEXT,
    college_name TEXT,
    degree TEXT,
    branch TEXT,
    graduation_year SMALLINT,
    career_goal TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);
```

```sql
-- Auto-create a profile row whenever a new auth user signs up
CREATE OR REPLACE FUNCTION handle_new_user()
RETURNS TRIGGER LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
    INSERT INTO public.profiles (id, email, full_name)
    VALUES (NEW.id, NEW.email, NEW.raw_user_meta_data ->> 'full_name');
    RETURN NEW;
END;
$$;

CREATE TRIGGER on_auth_user_created
    AFTER INSERT ON auth.users
    FOR EACH ROW EXECUTE FUNCTION handle_new_user();

-- Generic updated_at trigger, reused by every table below that has the column
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_profiles_updated_at
    BEFORE UPDATE ON profiles
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

*Note:* `career_goal` stays here as a lightweight, free-text field shown on the basic profile (useful even before Phase 3 exists). The structured version — `target_role` / `target_company` — lives in `career_profiles` once Phase 3 begins; the field is not duplicated across both tables.

### 5.2.2 documents

PDF files go into the private `documents` Storage bucket; this table holds metadata only.

```sql
CREATE TABLE documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    title TEXT,
    original_file_name TEXT NOT NULL,
    storage_path TEXT NOT NULL,
    file_type TEXT NOT NULL DEFAULT 'application/pdf',
    file_size_bytes BIGINT,
    file_hash TEXT NOT NULL, -- SHA-256
    status TEXT NOT NULL DEFAULT 'uploaded'
        CHECK (status IN ('uploaded','processing','ready','failed','deleted')),
    total_pages INTEGER,
    error_message TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE (user_id, file_hash) -- blocks duplicate uploads per user
);

CREATE TRIGGER trg_documents_updated_at
    BEFORE UPDATE ON documents
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

| Status | Meaning |
|---|---|
| `uploaded` | File received, no processing yet |
| `processing` | Text extraction and chunking in progress |
| `ready` | Embeddings stored, available for RAG queries |
| `failed` | Processing error — see `error_message` |
| `deleted` | Soft-deleted, not visible to the student |

## 5.3 Phase 1 — RAG Pipeline and Cited Chat

### 5.3.1 document_pages

Page-level text enables precise citations such as "page 7 of Operating Systems Notes" instead of a vague "from your document."

```
CREATE TABLE document_pages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    page_number INTEGER NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE (document_id, page_number)
);
```

The `UNIQUE(document_id, page_number)` constraint does double duty: it stops a retried extraction job from inserting the same page twice, and it is what makes the composite foreign key on `document_chunks` below possible.

### 5.3.2 document_chunks (most critical table)

The RAG system operates on chunks, not whole documents. **Design correction from the original draft:** the earlier schema stored a single `page_id` foreign key *and* duplicated `page_number` inside the `metadata` JSON. Because chunking uses 80–150 token overlap, a single chunk can legitimately span two pages — a scenario a single `page_id` cannot represent, and one where the FK and the JSON copy could silently disagree. The fix: replace `page_id` with real `start_page` / `end_page` columns, each backed by a composite foreign key into `document_pages`.

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index INTEGER NOT NULL,
    content TEXT NOT NULL,
    token_count INTEGER,
    start_page INTEGER NOT NULL,
    end_page INTEGER NOT NULL,
    embedding VECTOR(1536) NOT NULL, -- text-embedding-3-small
    metadata JSONB DEFAULT '{}'::jsonb,
    created_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE (document_id, chunk_index),
    FOREIGN KEY (document_id, start_page)
        REFERENCES document_pages (document_id, page_number),
    FOREIGN KEY (document_id, end_page)
        REFERENCES document_pages (document_id, page_number)
);
```

**Recommended chunking parameters:**

| Parameter | Value |
|---|---|
| Chunk size | 500–800 tokens |
| Overlap | 80–150 tokens |
| Preserve | page range and chunk order (now real columns, not JSON) |

**Example `metadata` JSONB** (soft, extensible data only — page numbers are no longer stored here):

```
{
  "section_title": "Deadlock Prevention",
  "start_char": 120,
  "end_char": 1850
}
```

Because the composite foreign keys require matching rows in `document_pages` to already exist, page rows must be inserted *before* chunk rows during ingestion — consistent with the processing order in §5.9.

### 5.3.3 conversations

```
CREATE TABLE conversations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    document_id UUID REFERENCES documents(id) ON DELETE SET NULL,
    title TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TRIGGER trg_conversations_updated_at
    BEFORE UPDATE ON conversations
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

`document_id` uses `ON DELETE SET NULL` rather than `CASCADE`: a conversation is a chat session, optionally scoped to one document, and should survive that document's deletion even if it loses the direct link.

### 5.3.4 messages

```
CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,
    role TEXT NOT NULL CHECK (role IN ('user','assistant','system')),
    content TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

### 5.3.5 message_sources (citation provenance)

```
CREATE TABLE message_sources (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    message_id UUID NOT NULL REFERENCES messages(id) ON DELETE CASCADE,
    chunk_id UUID REFERENCES document_chunks(id) ON DELETE SET NULL,
    similarity_score FLOAT,
```

```
    rank INTEGER,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

**Correction:** `chunk_id` is now nullable with `ON DELETE SET NULL`. In the original draft this column had no `ON DELETE` clause at all, meaning the implicit `RESTRICT` default would have made it permanently impossible to delete a document once any of its chunks had been cited in a chat answer. Chat history now survives chunk/document deletion, simply losing the live link to the source chunk while keeping the similarity score and rank for that turn.

This table is the proof, visible in the schema itself, that AI Scholar's answers are grounded in retrieved context rather than hallucinated — one of the most differentiated features in a student RAG project.

## 5.4 Phase 2 — Learning Tools Layer

### 5.4.1 quizzes

```
CREATE TABLE quizzes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    document_id UUID REFERENCES documents(id) ON DELETE SET NULL,
    title TEXT,
    topic TEXT,
    difficulty TEXT CHECK (difficulty IN ('easy','medium','hard')),
    question_count INTEGER,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

### 5.4.2 quiz_questions

```
CREATE TABLE quiz_questions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    quiz_id UUID NOT NULL REFERENCES quizzes(id) ON DELETE CASCADE,
    question_text TEXT NOT NULL,
    question_type TEXT NOT NULL
        CHECK (question_type IN ('mcq','true_false','fill_blank','short_answer')),
    options JSONB, -- array of choices for MCQ
    correct_answer TEXT NOT NULL,
    explanation TEXT,
    source_chunk_id UUID REFERENCES document_chunks(id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

### 5.4.3 quiz_attempts

```
CREATE TABLE quiz_attempts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    quiz_id UUID NOT NULL REFERENCES quizzes(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    score INTEGER,
    total_questions INTEGER,
    started_at TIMESTAMPTZ DEFAULT now(),
    completed_at TIMESTAMPTZ,
    CHECK (score IS NULL OR score <= total_questions)
);
```

### 5.4.4 quiz_answers

```
CREATE TABLE quiz_answers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    attempt_id UUID NOT NULL REFERENCES quiz_attempts(id) ON DELETE CASCADE,
    question_id UUID NOT NULL REFERENCES quiz_questions(id) ON DELETE CASCADE,
    selected_answer TEXT,
    is_correct BOOLEAN,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

### 5.4.5 study_plans

```
CREATE TABLE study_plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    title TEXT,
    goal TEXT,
    start_date DATE,
    end_date DATE,
    status TEXT NOT NULL DEFAULT 'active'
        CHECK (status IN ('active','completed','archived')),
    created_at TIMESTAMPTZ DEFAULT now()
);
```

### 5.4.6 study_plan_items

```
CREATE TABLE study_plan_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    study_plan_id UUID NOT NULL REFERENCES study_plans(id) ON DELETE CASCADE,
    document_id UUID REFERENCES documents(id) ON DELETE SET NULL,
    topic TEXT,
    scheduled_date DATE,
    estimated_minutes INTEGER,
    status TEXT NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending','in_progress','completed','skipped')),
    completed_at TIMESTAMPTZ
);
```

### 5.4.7 learning_progress

```
CREATE TABLE learning_progress (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    topic TEXT NOT NULL,
    progress_percentage INTEGER DEFAULT 0
        CHECK (progress_percentage BETWEEN 0 AND 100),
    last_studied_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE (user_id, document_id, topic)
);

CREATE TRIGGER trg_learning_progress_updated_at
    BEFORE UPDATE ON learning_progress
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

`document_id` is `NOT NULL` here (progress is always tracked against a specific document's topic) with `ON DELETE CASCADE`, and the `UNIQUE` constraint makes topic progress an upsert target rather than an ever-growing log table.

**Capabilities unlocked by Phase 2 tables:** topic completion and coverage percentage, quiz performance trends over time, study streaks and daily revision goals, weak-topic detection, personalized semester-level schedules.

## 5.5 Phase 3 — Career and Profile Module

All career tables connect to academic data through the shared `user_id` foreign key into `profiles`.

### 5.5.1 career_profiles

```
CREATE TABLE career_profiles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL UNIQUE REFERENCES profiles(id) ON DELETE CASCADE,
    headline TEXT,
    target_role TEXT,
    target_company TEXT,
    linkedin_url TEXT,
    github_url TEXT,
    resume_document_id UUID REFERENCES documents(id) ON DELETE SET NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TRIGGER trg_career_profiles_updated_at
    BEFORE UPDATE ON career_profiles
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

**Correction:** the original draft repeated a `career_goal` field already present on `profiles`. It is removed here — `profiles.career_goal` remains the single free-text field; `target_role` / `target_company` are this table's structured equivalent, and `UNIQUE(user_id)` enforces the intended one-profile-per-student model.

### 5.5.2 user_skills

```
CREATE TABLE user_skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    skill_name TEXT NOT NULL,
    category TEXT,
    proficiency_level TEXT
        CHECK (proficiency_level IN ('beginner','intermediate','advanced')),
    evidence TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    UNIQUE (user_id, skill_name)
);
```

`UNIQUE(user_id, skill_name)` stops repeated skill-extraction runs from accumulating duplicate rows for the same skill.

### 5.5.3 target_roles

```
CREATE TABLE target_roles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    role_name TEXT NOT NULL,
    company_name TEXT,
    job_description TEXT,
    required_skills JSONB,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

#### 5.5.4 gap_analyses

```
CREATE TABLE gap_analyses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    target_role_id UUID REFERENCES target_roles(id) ON DELETE SET NULL,
    overall_score FLOAT CHECK (overall_score BETWEEN 0 AND 100),
    matched_skills JSONB,
    missing_skills JSONB,
    recommendations TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

`target_role_id` uses `ON DELETE SET NULL`: the matched/missing skill snapshot in the JSONB columns remains meaningful even if the target role record is later edited or removed.

### 5.6 Phase 4 — Integration View

Phase 4 is application logic and UI over data that already exists; the only schema artifact is a read-only view assembling the unified student profile consumed by the dashboard and the "digital mentor" thread.

```
CREATE VIEW student_overview AS
SELECT
    p.id AS user_id,
    p.full_name,
    p.career_goal,
    COUNT(DISTINCT d.id) AS documents_uploaded,
    COUNT(DISTINCT qa.id) AS quiz_attempts_taken,
    ROUND(AVG(qa.score::NUMERIC / NULLIF(qa.total_questions, 0)) * 100, 1)
        AS avg_quiz_score_pct,
    COUNT(DISTINCT lp.id) AS topics_tracked,
    cp.target_role,
    cp.target_company
FROM profiles p
LEFT JOIN documents d ON d.user_id = p.id AND d.status != 'deleted'
LEFT JOIN quiz_attempts qa ON qa.user_id = p.id
LEFT JOIN learning_progress lp ON lp.user_id = p.id
LEFT JOIN career_profiles cp ON cp.user_id = p.id
GROUP BY p.id, p.full_name, p.career_goal, cp.target_role, cp.target_company;

-- Postgres 15+: make the view respect the caller's RLS, not the view owner's
ALTER VIEW student_overview SET (security_invoker = true);
```

## 5.7 Row-Level Security

**Non-negotiable:** every table holding student data has RLS enabled *and* a policy. Enabling RLS alone denies all access by default for any non-service-role connection — the original draft enabled RLS on five tables but only wrote policies for two, which would have silently broken every query against the other three the moment RLS was switched on.

### 5.7.1 Enable RLS on every user-owned table

```
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE document_pages ENABLE ROW LEVEL SECURITY;
ALTER TABLE document_chunks ENABLE ROW LEVEL SECURITY;
ALTER TABLE conversations ENABLE ROW LEVEL SECURITY;
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE message_sources ENABLE ROW LEVEL SECURITY;
ALTER TABLE quizzes ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz_questions ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz_attempts ENABLE ROW LEVEL SECURITY;
ALTER TABLE quiz_answers ENABLE ROW LEVEL SECURITY;
ALTER TABLE study_plans ENABLE ROW LEVEL SECURITY;
ALTER TABLE study_plan_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE learning_progress ENABLE ROW LEVEL SECURITY;
ALTER TABLE career_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_skills ENABLE ROW LEVEL SECURITY;
ALTER TABLE target_roles ENABLE ROW LEVEL SECURITY;
ALTER TABLE gap_analyses ENABLE ROW LEVEL SECURITY;
```

### 5.7.2 Pattern A — tables with a direct `user_id` column

```
CREATE POLICY own_profile_only ON profiles
    FOR ALL USING (auth.uid() = id) WITH CHECK (auth.uid() = id);

CREATE POLICY own_documents_only ON documents
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_conversations_only ON conversations
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_quizzes_only ON quizzes
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_quiz_attempts_only ON quiz_attempts
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_study_plans_only ON study_plans
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_learning_progress_only ON learning_progress
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_career_profile_only ON career_profiles
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_skills_only ON user_skills
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
```

```
CREATE POLICY own_target_roles_only ON target_roles
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY own_gap_analyses_only ON gap_analyses
    FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);
```

### 5.7.3 Pattern B — tables reached only through a foreign-key chain

These tables have no direct `user_id` column, so ownership is checked through the parent table.

```
CREATE POLICY own_pages_only ON document_pages
    FOR ALL USING (
        document_id IN (SELECT id FROM documents WHERE user_id = auth.uid())
    ) WITH CHECK (
        document_id IN (SELECT id FROM documents WHERE user_id = auth.uid())
    );

CREATE POLICY own_chunks_only ON document_chunks
    FOR ALL USING (
        document_id IN (SELECT id FROM documents WHERE user_id = auth.uid())
    ) WITH CHECK (
        document_id IN (SELECT id FROM documents WHERE user_id = auth.uid())
    );

CREATE POLICY own_messages_only ON messages
    FOR ALL USING (
        conversation_id IN (SELECT id FROM conversations WHERE user_id = auth.uid())
    ) WITH CHECK (
        conversation_id IN (SELECT id FROM conversations WHERE user_id = auth.uid())
    );

CREATE POLICY own_message_sources_only ON message_sources
    FOR ALL USING (
        message_id IN (
            SELECT m.id FROM messages m
            JOIN conversations c ON c.id = m.conversation_id
            WHERE c.user_id = auth.uid()
        )
    );

CREATE POLICY own_quiz_questions_only ON quiz_questions
    FOR ALL USING (
        quiz_id IN (SELECT id FROM quizzes WHERE user_id = auth.uid())
    );

CREATE POLICY own_quiz_answers_only ON quiz_answers
    FOR ALL USING (
        attempt_id IN (SELECT id FROM quiz_attempts WHERE user_id = auth.uid())
    );

CREATE POLICY own_study_plan_items_only ON study_plan_items
    FOR ALL USING (
        study_plan_id IN (SELECT id FROM study_plans WHERE user_id = auth.uid())
    );
```

#### 5.7.4 Additional security rules

- Use the Supabase service-role key only on the backend, never in frontend code.
- Access uploaded PDFs through time-limited signed URLs, not direct public links.
- Because the FastAPI backend will typically call `match_chunks` using the service-role key (which bypasses RLS), `filter_user_id` becomes the *only* protection against cross-user leakage for that call. It must always come from the authenticated session server-side, never from client-supplied input.

### 5.8 Indexes and Vector Search

#### 5.8.1 Standard relational indexes

Postgres does not automatically index foreign-key columns. The original draft indexed 7 of the relevant tables; the list below covers every foreign key used in a join, cascade, or RLS subquery.

```
CREATE INDEX idx_documents_user_status ON documents (user_id, status);
CREATE INDEX idx_pages_document_page ON document_pages (document_id, page_number);
CREATE INDEX idx_chunks_document_index ON document_chunks (document_id, chunk_index);
CREATE INDEX idx_chunks_start_page ON document_chunks (document_id, start_page);
CREATE INDEX idx_conversations_user ON conversations (user_id);
CREATE INDEX idx_messages_conversation_time ON messages (conversation_id, created_at)
    ;
CREATE INDEX idx_message_sources_message ON message_sources (message_id);
CREATE INDEX idx_quizzes_user_document ON quizzes (user_id, document_id);
CREATE INDEX idx_quiz_questions_quiz ON quiz_questions (quiz_id);
CREATE INDEX idx_quiz_attempts_quiz_user ON quiz_attempts (quiz_id, user_id);
CREATE INDEX idx_quiz_answers_attempt ON quiz_answers (attempt_id);
CREATE INDEX idx_study_plans_user ON study_plans (user_id);
CREATE INDEX idx_study_plan_items_plan ON study_plan_items (study_plan_id);
CREATE INDEX idx_learning_progress_user ON learning_progress (user_id);
CREATE INDEX idx_career_profiles_user ON career_profiles (user_id);
CREATE INDEX idx_user_skills_user ON user_skills (user_id);
CREATE INDEX idx_target_roles_user ON target_roles (user_id);
CREATE INDEX idx_gap_analyses_user ON gap_analyses (user_id);
CREATE INDEX idx_gap_analyses_target_role ON gap_analyses (target_role_id);
```

#### 5.8.2 HNSW vector index for semantic search

```
CREATE INDEX idx_chunks_embedding_hnsw
    ON document_chunks USING hnsw (embedding vector_cosine_ops);
```

Use an exact (flat) scan during early development with a small chunk count; switch to HNSW once a few thousand chunks accumulate, since it trades a small recall penalty for materially faster search.

#### 5.8.3 Vector search RPC function

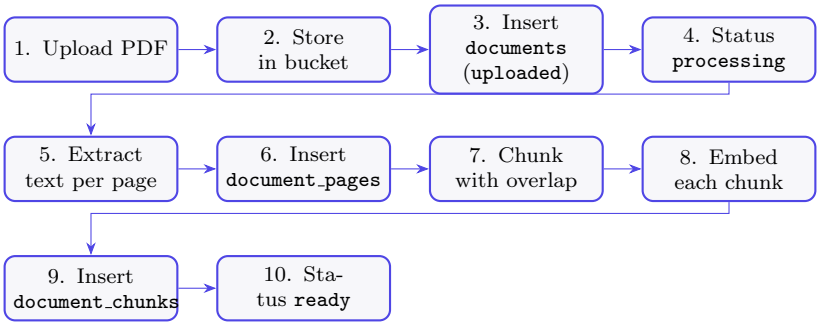
```
CREATE OR REPLACE FUNCTION match_chunks(
    query_embedding VECTOR(1536),
    match_count INTEGER,
    filter_user_id UUID,
    filter_doc_ids UUID[] DEFAULT NULL
)
RETURNS TABLE (
```

```
    id UUID,
    document_id UUID,
    content TEXT,
    start_page INTEGER,
    end_page INTEGER,
    metadata JSONB,
    similarity_score FLOAT
)
LANGUAGE plpgsql SECURITY INVOKER AS $$
BEGIN
    RETURN QUERY
    SELECT
        dc.id, dc.document_id, dc.content,
        dc.start_page, dc.end_page, dc.metadata,
        1 - (dc.embedding <=> query_embedding) AS similarity_score
    FROM document_chunks dc
    JOIN documents d ON dc.document_id = d.id
    WHERE d.user_id = filter_user_id
      AND (filter_doc_ids IS NULL OR dc.document_id = ANY(filter_doc_ids))
    ORDER BY dc.embedding <=> query_embedding
    LIMIT match_count;
END;
$$;
```

**Corrections from the original draft:** the function now returns `start_page`/`end_page` so the calling code can render an accurate citation range; `filter_doc_ids` defaults to `NULL` and is treated as "search all of this user's documents" rather than requiring the caller to always pass an explicit array; `SECURITY INVOKER` is stated explicitly so RLS still applies on top of the `filter_user_id` check when called with a non-service-role connection.

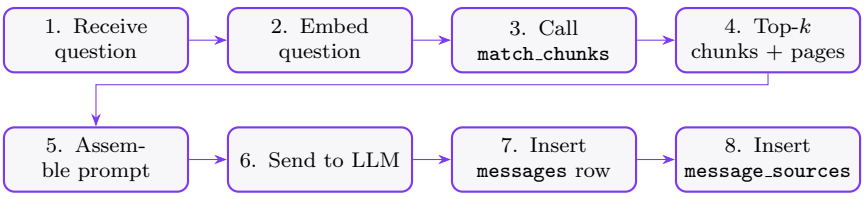
## 5.9 System Workflows

### 5.9.1 PDF Upload and Processing Flow




Step 6 must complete before step 9, because each chunk's `start_page`/`end_page` is a composite foreign key into `document_pages`.

### 5.9.2 RAG Query Flow (Student Asks a Question)



The final response returns the answer with inline citations resolved from `start_page`/`end_page` on each cited chunk.

## 5.10 Implementation Order

1. Create the Supabase project; enable the `pgvector` extension.
2. Configure Supabase Auth; create the private `documents` storage bucket.
3. Create `profiles`, the `handle_new_user` trigger, and the shared `set_updated_at` function.
4. Create `documents` with its status `CHECK` constraint and `(user_id, file_hash)` uniqueness.
5. Create `document_pages` with its `UNIQUE(document_id, page_number)` constraint.
6. Create `document_chunks` with the `vector(1536)` column and the composite foreign keys into `document_pages`.
7. Apply Row-Level Security — enable *and* write the policy — on every table created so far, before writing any application code that reads them.
8. Create the `match_chunks` RPC function.
9. Add the HNSW index on `document_chunks.embedding` once meaningful data volume exists.
10. Create `conversations`, `messages`, `message_sources`; apply their RLS policies.
11. Add all standard relational indexes from §5.9.
12. Test the complete Phase 1 flow end-to-end: upload → chunk → embed → retrieve → cited answer.
13. Add Phase 2 quiz and study-planner tables, each with its RLS policy and indexes, before writing Phase 2 application logic.
14. Add Phase 3 career tables, each with its RLS policy and indexes.
15. Add the `student_overview` view for Phase 4.

## 5.11 Out of Scope for Phase 0–3

**Do not build yet,** however compelling, until the RAG core and the modules above it are proven end-to-end:

- Handwritten-notes OCR ingestion.
- Lecture audio / transcription ingestion.
- GitHub repository analysis, LeetCode/Codeforces profile scraping.
- Advanced recommendation engines; complex career-readiness scoring algorithms.
- Flashcard / spaced-repetition scheduling algorithms.
- Multi-document collection groups.

**Forward-compatibility notes for Phase 5** (so it does not require destructive migrations later): `documents` can gain an `ingestion_method` column (`typed` / `ocr` / `transcript`) without breaking existing rows, since it would default to `typed`; `document_pages` can similarly gain a `source_type` column. Neither is created now.

### Phase 1 Success Criteria

- Secure user account via Supabase Auth, with an auto-created profile.
- PDF upload to private storage; metadata in `documents`.
- Page-level text in `document_pages`.
- Chunked, embedded text in `document_chunks` with accurate page ranges.
- Semantic retrieval via `match_chunks`.
- Cited AI answers with `message_sources` resolving to real page ranges.

## 5.12 Full Schema Reference Summary

| Table / View | Phase | Purpose |
|---|---|---|
| `profiles` | 0 | User identity; extends `auth.users`; auto-created on signup |
| `documents` | 0 | PDF file metadata and processing status |
| `document_pages` | 1 | Per-page text for accurate citations |
| `document_chunks` | 1 | Chunked text with embeddings and real page-range columns (core RAG) |
| `conversations` | 1 | Chat session grouping per user/document |
| `messages` | 1 | Individual user and assistant turns |
| `message_sources` | 1 | Chunk provenance per assistant answer |
| `quizzes` | 2 | Quiz metadata generated from documents |
| `quiz_questions` | 2 | Questions linked to source chunks |
| `quiz_attempts` | 2 | Student attempt records with scores |
| `quiz_answers` | 2 | Per-question selected answers |
| `study_plans` | 2 | Goal-based study plans |
| `study_plan_items` | 2 | Per-topic scheduled tasks |
| `learning_progress` | 2 | Topic-level progress tracking (upserted) |
| `career_profiles` | 3 | Career goals, links, resume reference (one per user) |
| `user_skills` | 3 | Identified skills with proficiency levels |
| `target_roles` | 3 | Target jobs with required-skill mappings |
| `gap_analyses` | 3 | Skill-gap scoring and recommendations |
| `student_overview` | 4 | Read-only view unifying academic + career data |

# 6 API Specification

This section defines the FastAPI contract layer between the Next.js frontend and the schema in §5. Its purpose is narrow but specific: **eliminate integration errors** between frontend, backend, and database by making every request/response field name, type, and enum value trace back to a single column in §5 rather than being redefined independently on each side of the stack. Appendix B contains the full field-parity tables proving this for every resource.

## 6.1 Conventions

- **Base path:** `/v1`. All paths below are relative to it (e.g. `/v1/profile`).
- **Auth:** every endpoint requires `Authorization: Bearer <supabase_jwt>`. The backend extracts `auth.uid()` from the verified JWT — it is never accepted as a client-supplied field, since `filter_user_id` in `match_chunks` and every RLS policy in §5 depends on this value being trustworthy.
- **IDs:** every `id` and foreign-key field is a UUID, serialized as a JSON string. No endpoint uses integer IDs, matching every primary key in the schema.
- **Timestamps:** every `TIMESTAMPTZ` column is serialized as an ISO 8601 string with an explicit offset (e.g. `"2026-06-28T10:15:00Z"`). The backend never serializes naive datetimes — this is the wire-format consequence of the §5 decision to use `TIMESTAMPTZ` everywhere.
- **Enums:** every field backed by a `CHECK (... IN (...))` constraint uses one shared Python `Enum` per field, imported by both the Pydantic request model and the Pydantic response model. The valid values are never re-typed as string literals in more than one place in the backend codebase — this is what prevents the API and the database from silently drifting apart on, e.g., what counts as a valid `quiz_questions.question_type`.
- **Server-managed fields:** `id`, `user_id`, `created_at`, `updated_at`, and any column with a database-computed value (e.g. `quiz_attempts.score`, `quiz_answers.is_correct`) are never accepted in a request body. If a client sends one anyway, the backend returns `422` with `error.code = "read_only_field"` rather than silently ignoring it — silent ignoring is exactly the kind of mismatch that causes integration bugs to surface only in production.
- **Pagination envelope** (all `GET` list endpoints):

```
{
  "data": [ ... ],
  "page": 1,
  "page_size": 20,
  "total": 137
}
```

- **Error envelope** (all non-2xx responses):

```
{
  "error": {
    "code": "string",
    "message": "string",
    "details": {}
  }
}
```

- **Standard status codes used throughout:** 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 503 Upstream Service Unavailable (embedding/LLM provider failure).

## 6.2 Phase 0 — Auth and Profile

```
GET /v1/profile
```

Returns the caller's own `profiles` row. Every field maps 1:1 to a column — see Appendix B.1.

```
PATCH /v1/profile
```

Body accepts any subset of `full_name`, `avatar_url`, `college_name`, `degree`, `branch`, `graduation_year`, `career_goal`. `email` is excluded — it is changed through Supabase Auth directly, never through this table, so the API does not expose it as writable even though the column exists.

## 6.3 Phase 1 — Documents and RAG Chat

```
POST /v1/documents (multipart/form-data: file)
```

The backend computes `file_hash` (SHA-256) and `file_size_bytes` server-side, uploads the file to the private Storage bucket, and inserts a `documents` row with `status = 'uploaded'`. `storage_path` and `file_hash` are never returned to the client — they are internal-only fields; a separate signed URL is issued on demand instead (§5.7.4 of the schema). If the computed hash collides with an existing `(user_id, file_hash)` pair, the unique constraint on `documents` fires and the backend translates the resulting database error into:

```
HTTP 409
{
  "error": {
    "code": "duplicate_document",
    "message": "This file has already been uploaded.",
    "details": { "existing_document_id": "..." }
  }
}
```

This is a direct example of why the schema constraint and the API error code must be designed together: without this translation step, the client would receive an unhandled 500 from a raw Postgres unique-violation.

```
GET /v1/documents
GET /v1/documents/{document_id}
DELETE /v1/documents/{document_id}
GET /v1/documents/{document_id}/pages?page=&page_size=
```

`DELETE` performs a soft delete (`status = 'deleted'`), it does not run a SQL `DELETE` — this keeps the endpoint idempotent and avoids triggering the `ON DELETE CASCADE` chain through `document_pages`/`document_chunks` from a single user action. A genuine hard delete is an admin-only operation, not exposed here. A document not owned by the caller returns 404, not 403 — RLS would block the row at the database layer regardless, and the API deliberately does not reveal whether the ID exists for another user.

```
POST /v1/conversations
GET /v1/conversations
GET /v1/conversations/{conversation_id}/messages
```

`POST` body: `document_id` (nullable) and `title` (nullable) — matching the nullable columns on `conversations`. The messages endpoint returns each message together with its resolved `message_sources`, joined to `document_chunks` for the page range:

```
{
  "data": [
    {
      "id": "9f1c...",
      "role": "assistant",
      "content": "Deadlock prevention requires ...",
      "created_at": "2026-06-28T10:15:32Z",
      "sources": [
        {
          "chunk_id": "a31e...",
          "document_id": "77bd...",
          "start_page": 4,
          "end_page": 5,
          "similarity_score": 0.86,
          "rank": 1
        }
      ]
    }
  ]
}
```

```
POST /v1/conversations/{conversation_id}/messages
```

The core RAG endpoint. Request body contains only `content` — `role` is never client-supplied; the backend always inserts the user turn with `role = 'user'` and the generated turn with `role = 'assistant'`, matching the three-value `CHECK` constraint on `messages.role` (the client literally cannot construct a request that violates it, because the field isn't exposed).

```
Request:
{ "content": "Explain deadlock prevention from my notes." }

Response 201:
{
  "user_message": {
    "id": "c1a4...", "role": "user",
    "content": "Explain deadlock prevention from my notes.",
    "created_at": "2026-06-28T10:15:30Z"
  },
  "assistant_message": {
    "id": "9f1c...", "role": "assistant",
    "content": "Deadlock prevention requires ...",
    "created_at": "2026-06-28T10:15:32Z",
    "sources": [
      { "chunk_id": "a31e...", "document_id": "77bd...",
        "start_page": 4, "end_page": 5,
        "similarity_score": 0.86, "rank": 1 }
    ]
  }
}
```

Server flow, matching §5.9.2 exactly: embed `content` with `text-embedding-3-small` → call `match_chunks(query_embedding, match_count, filter_user_id, filter_doc_ids)` with `filter_user_id` from the verified JWT and `filter_doc_ids` taken from the conversation's `document_id` (or `NULL`

to search all of the user's documents) → assemble the prompt → call the Anthropic API → insert `messages` (`role='assistant'`) → insert one `message_sources` row per retrieved chunk. If the embedding or generation call fails, the backend returns `503` with `error.code = "llm_unavailable"` and does *not* insert a partial assistant message — avoiding an orphaned row that a retry would duplicate.

## 6.4 Phase 2 — Quizzes and Study Planner

```
POST /v1/quizzes
GET /v1/quizzes/{quiz_id}
```

`POST` body: `document_id` (nullable), `topic` (nullable), `difficulty` (nullable, one of `easy`/`medium`/`hard` — same three values as the `quizzes.difficulty CHECK`), `question_count`. The backend retrieves context via `match_chunks`, generates questions, and inserts the `quizzes` row followed by its `quiz_questions` rows, with `question_type` restricted to the same four values as `quiz_questions.question_type`. `GET` omits `correct_answer` and `explanation` from each question *by default* — this is an API-layer access-control decision, not a schema one, made so the frontend cannot trivially reveal answers before an attempt is submitted; pass `?reveal=true` after `completed_at` is set on the relevant attempt to include them.

```
POST /v1/quizzes/{quiz_id}/attempts
POST /v1/quiz-attempts/{attempt_id}/answers
POST /v1/quiz-attempts/{attempt_id}/complete
```

Starting an attempt inserts a `quiz_attempts` row with `started_at = now()` and `score = NULL`. Submitting an answer inserts a `quiz_answers` row, with `is_correct` computed server-side by comparing `selected_answer` to `quiz_questions.correct_answer` — never trusted from the client. Completing an attempt computes `score` as the count of `is_correct = true` rows and sets `completed_at = now()`; the backend computes this *before* the update so it can never violate the `CHECK (score <= total_questions)` constraint on `quiz_attempts` — the constraint is a safety net, not the primary validation path.

```
POST /v1/study-plans
GET /v1/study-plans/{study_plan_id}
POST /v1/study-plans/{study_plan_id}/items
PATCH /v1/study-plan-items/{item_id}
```

`study_plans.status` and `study_plan_items.status` are never settable on creation — they default to `'active'` and `'pending'` respectively, exactly matching the schema defaults. `PATCH` on an item accepts `status` only from the four values in `study_plan_items.status CHECK`; any other string returns `422` from the Pydantic model before the request ever reaches the database, rather than surfacing a Postgres constraint-violation error.

```
PUT /v1/learning-progress
```

Deliberately `PUT`, not `POST`: the endpoint is an upsert keyed on `(user_id, document_id, topic)`, mirroring the `UNIQUE` constraint on `learning_progress` exactly. The backend issues `INSERT ... ON CONFLICT (user_id, document_id, topic) DO UPDATE`, so calling this endpoint repeatedly for the same topic is always safe and never produces duplicate rows or a `409`.

## 6.5 Phase 3 — Career Module

```
GET /v1/career-profile
PUT /v1/career-profile
```

Also an upsert (`PUT`), matching `UNIQUE(user_id)` on `career_profiles` — there is exactly one career profile per student, so create and update are the same operation at the API layer, just as they are the same constraint at the schema layer. `GET` returns `404` if the row does not exist yet (it is not auto-created, unlike `profiles`). If `resume_document_id` is supplied, the backend validates that the referenced document belongs to the caller *and* has `status = 'ready'` before writing it; otherwise it returns `422` with `error.code = "invalid_resume_document"` rather than allowing a write that the `ON DELETE SET NULL` foreign key would silently tolerate but that makes no product sense.

```
GET /v1/skills
POST /v1/skills
PATCH /v1/skills/{skill_id}
```

`POST` maps to `(user_id, skill_name)` in `user_skills`; a duplicate `skill_name` for the same user returns `409` (translating the unique-violation) with a message pointing the client at `PATCH` instead. `PATCH` accepts `category`, `proficiency_level` (one of `beginner`/`intermediate`/`advanced`), and `evidence` — `skill_name` is immutable through this endpoint, since it is the identity half of the unique constraint.

```
POST /v1/target-roles
GET /v1/target-roles
POST /v1/gap-analyses
GET /v1/gap-analyses/{gap_analysis_id}
```

`target_roles.required_skills` and `gap_analyses.matched_skills`/`missing_skills` are `JSONB` columns with no fixed shape at the database level; the API documents (but does not enforce in Postgres) a consistent array-of-objects shape so the frontend always knows what to render:

```
"required_skills": [
  { "skill": "React", "level": "intermediate" },
  { "skill": "PostgreSQL", "level": "beginner" }
]
```

`POST /v1/gap-analyses` takes only `target_role_id`; the backend computes `matched_skills`, `missing_skills`, and `overall_score` (clamped to the `0--100 CHECK` range) by diffing the caller's `user_skills` against the target role's `required_skills` — none of these derived fields are ever accepted as request input.

## 6.6 Phase 4 — Integration

```
GET /v1/overview
```

A direct, read-only pass-through of the `student_overview` view (§5.6). Because the view is declared `security_invoker`, the backend must call it using the caller's own authenticated connection (or explicitly pass `filter_user_id` if called via service role) — calling it through the service-role key without an explicit filter would return every student's row, not just the caller's.

## 6.7 Endpoint Summary

| Method | Path | Phase | Primary Table(s) |
|---|---|---|---|
| GET | `/v1/profile` | 0 | `profiles` |
| PATCH | `/v1/profile` | 0 | `profiles` |
| POST | `/v1/documents` | 1 | `documents` |

| Method | Path | Phase | Primary Table(s) |
|---|---|---|---|
| GET | /v1/documents | 1 | documents |
| GET | /v1/documents/{id} | 1 | documents |
| DELETE | /v1/documents/{id} | 1 | documents |
| GET | /v1/documents/{id}/pages | 1 | document_pages |
| POST | /v1/conversations | 1 | conversations |
| GET | /v1/conversations | 1 | conversations |
| GET | /v1/conversations/{id}/messages | 1 | messages, message_sources |
| POST | /v1/conversations/{id}/messages | 1 | messages, message_sources, document_chunks (via match_chunks) |
| POST | /v1/quizzes | 2 | quizzes, quiz_questions |
| GET | /v1/quizzes/{id} | 2 | quizzes, quiz_questions |
| POST | /v1/quizzes/{id}/attempts | 2 | quiz_attempts |
| POST | /v1/quiz-attempts/{id}/answers | 2 | quiz_answers |
| POST | /v1/quiz-attempts/{id}/complete | 2 | quiz_attempts |
| POST | /v1/study-plans | 2 | study_plans |
| GET | /v1/study-plans/{id} | 2 | study_plans, study_plan_items |
| POST | /v1/study-plans/{id}/items | 2 | study_plan_items |
| PATCH | /v1/study-plan-items/{id} | 2 | study_plan_items |
| PUT | /v1/learning-progress | 2 | learning_progress |
| GET | /v1/career-profile | 3 | career_profiles |
| PUT | /v1/career-profile | 3 | career_profiles |
| GET | /v1/skills | 3 | user_skills |
| POST | /v1/skills | 3 | user_skills |
| PATCH | /v1/skills/{id} | 3 | user_skills |
| POST | /v1/target-roles | 3 | target_roles |
| GET | /v1/target-roles | 3 | target_roles |
| POST | /v1/gap-analyses | 3 | gap_analyses |
| GET | /v1/gap-analyses/{id} | 3 | gap_analyses |
| GET | /v1/overview | 4 | student_overview |

## Appendix A — Reconciliation Changelog

Changes applied while merging the original project-plan and schema-design drafts into this document:

1. **Phase renumbering.** The schema's original Phase 1 (quiz) / Phase 2 (study planner) / Phase 3 (career) numbering is replaced with the plan's own numbering, where Phase 2 ("Learning Tools Layer") covers both quizzes and the study planner, and Phase 3 is career. The plan and schema now always refer to the same phase number for the same feature.
2. `document_chunks` **page reference.** Removed the `page_id` foreign key and the duplicated `page_number` inside `metadata`, which could disagree for chunks spanning a page boundary. Replaced with `start_page`/`end_page` columns backed by composite foreign keys into `document_pages`.
3. **Embedding dimension finalized.** Locked to OpenAI `text-embedding-3-small`, `vector(1536)`, as a Phase 0 decision rather than a deferred one, since `pgvector` cannot change dimension after data exists without dropping and re-embedding.
4. **Row-Level Security completed.** Every table with RLS enabled now has an explicit policy. The original draft enabled RLS on five tables but only supplied policies for two.
5. `ON DELETE` **behavior added throughout.** `message_sources.chunk_id`, `quiz_questions.source_chunk_id`, `quizzes.document_id`, `study_plan_items.document_id`, and `career_profiles.resume_document_id` previously had no `ON DELETE` clause, which defaults to `RESTRICT` and would have made documents undeletable once referenced. All now use `SET NULL` or `CASCADE` as appropriate.
6. **Profile auto-creation trigger added.** A trigger on `auth.users` now creates the matching `profiles` row on signup; none existed in the original draft.
7. `updated_at` **triggers added** for `profiles`, `documents`, `conversations`, `career_profiles`, and `learning_progress`.
8. `TIMESTAMP` **changed to** `TIMESTAMPTZ` throughout, for unambiguous time-zone handling.
9. `CHECK` **constraints added** for all previously free-text status/role/type fields (`documents.status`, `messages.role`, `quizzes.difficulty`, `quiz_questions.question_type`, `study_plans.status`, `study_plan_items.status`, `user_skills.proficiency_level`, plus numeric range checks on `learning_progress.progress_percentage`, `gap_analyses.overall_score`, and `quiz_attempts.score`).
10. **Missing uniqueness constraints added:** `document_pages(document_id, page_number)`, `career_profiles(user_id)`, `user_skills(user_id, skill_name)`, `learning_progress(user_id, document_id, topic)`.
11. **Duplicate field removed.** `career_profiles.career_goal` duplicated `profiles.career_goal`; removed in favor of the more structured `target_role`/`target_company` pair.
12. **Index coverage completed.** Added indexes for every foreign key used in a join, cascade, or RLS subquery that the original draft left unindexed (`message_sources`, `quiz_questions`, `quiz_attempts`, `quiz_answers`, `study_plan_items`, `learning_progress`, and all Phase 3 tables).
13. `match_chunks` **updated** to return page-range columns, accept a `NULL` `filter_doc_ids` as "search all documents," and explicitly declare `SECURITY INVOKER`.

14. `student_overview` **view added** as the concrete schema artifact for Phase 4, set to `security_invoker` so it respects the querying user's RLS rather than the view owner's.

# Appendix B — Endpoint-to-Schema Field Parity

Each table below lists every field an endpoint sends or returns for a given resource, mapped directly to its database column, type, and constraint. This is the artifact that makes integration errors structurally hard to introduce: if a frontend field, a Pydantic field, and a database column ever disagree, the disagreement is visible in a single row here rather than discovered at request time.

## B.1 profiles

| JSON field | DB column | Type | Notes |
|---|---|---|---|
| `id` | `id` | UUID | read-only; equals `auth.uid()` |
| `full_name` | `full_name` | string/null | writable via `PATCH` |
| `email` | `email` | string/null | read-only here; changed via Supabase Auth |
| `avatar_url` | `avatar_url` | string/null | writable |
| `college_name` | `college_name` | string/null | writable |
| `degree` | `degree` | string/null | writable |
| `branch` | `branch` | string/null | writable |
| `graduation_year` | `graduation_year` | integer/null | writable; `SMALLINT` range |
| `career_goal` | `career_goal` | string/null | writable; free text only (see §5.5.1) |
| `created_at` | `created_at` | ISO 8601 | read-only |
| `updated_at` | `updated_at` | ISO 8601 | read-only; server sets via trigger |

## B.2 documents

| JSON field | DB column | Type | Notes |
|---|---|---|---|
| `id` | `id` | UUID | read-only |
| `title` | `title` | string/null | writable on upload only |
| `original_file_name` | `original_file_name` | string | read-only; from upload |
| `file_type` | `file_type` | string | read-only; default `application/pdf` |
| `file_size_bytes` | `file_size_bytes` | integer | read-only |
| `status` | `status` | enum | read-only; one of the 5 values in §5.2.2 |
| `total_pages` | `total_pages` | integer/null | read-only; set after processing |
| `error_message` | `error_message` | string/null | read-only; populated on `failed` |
| `created_at` / `updated_at` | same | ISO 8601 | read-only |
| — (not exposed) | `storage_path` | string | internal only; signed URL issued separately |
| — (not exposed) | `file_hash` | string | internal only; duplicate-detection key |

### B.3 conversations / messages / message_sources

| JSON field | DB column | Type | Notes |
|---|---|---|---|
| document_id | conversations.document_id | UUID/null | writable on create only |
| title | conversations.title | string/null | writable |
| role | messages.role | enum | never client-writable; server sets 'user'/'assistant' |
| content | messages.content | string | client-writable only on the user turn |
| sources[].chunk_id | message_sources.chunk_id | UUID/null | read-only; may be null if source chunk later deleted |
| sources[].similarity_score | message_sources.similarity_score | float | read-only |
| sources[].rank | message_sources.rank | integer | read-only |
| sources[].start_page / end_page | document_chunks.start_page / end_page | integer | read-only; resolved via join, not stored on message_sources |

### B.4 quizzes / quiz_questions / quiz_attempts / quiz_answers

| JSON field | DB column | Type | Notes |
|---|---|---|---|
| difficulty | quizzes.difficulty | enum/null | one of easy/medium/hard |
| question_type | quiz_questions.question_type | enum | one of mcq/true_false/fill_blank/short_answer |
| options | quiz_questions.options | JSON array/null | MCQ choices only |
| correct_answer, explanation | same columns | string | omitted from GET unless ?reveal=true post-completion |
| score, total_questions | quiz_attempts.score, .total_questions | integer | read-only; server-computed, score <= total_questions |
| selected_answer | quiz_answers.selected_answer | string | client-writable |
| is_correct | quiz_answers.is_correct | boolean | read-only; server-computed |

### B.5 study_plans / study_plan_items / learning_progress

| JSON field | DB column | Type | Notes |
|---|---|---|---|
| `status` (plan) | `study_plans.status` | enum | one of `active`/`completed`/`archived`; default-only on create |
| `status` (item) | `study_plan_items.status` | enum | one of `pending`/`in_progress`/`completed`/`skipped` |
| `progress_percentage` | `learning_progress.progress_percentage` | integer | range 0–100, enforced both in Pydantic and the DB `CHECK` |
| `(user_id, document_id, topic)` | `learning_progress` unique key | — | this is why `PUT` (upsert) is used, not `POST` |

### B.6 career_profiles / user_skills / target_roles / gap_analyses

| JSON field | DB column | Type | Notes |
|---|---|---|---|
| `resume_document_id` | `career_profiles.resume_document_id` | UUID/null | validated against caller's own `documents` with `status='ready'` before write |
| `proficiency_level` | `user_skills.proficiency_level` | enum/null | one of `beginner`/`intermediate`/`advanced` |
| `(user_id, skill_name)` | `user_skills` unique key | — | duplicate `POST` returns `409`, not a silent upsert |
| `required_skills` | `target_roles.required_skills` | JSONB | documented array-of-objects shape; not DB-enforced |
| `overall_score` | `gap_analyses.overall_score` | float | range 0–100; always server-computed |
| `matched_skills`, `missing_skills` | same columns | JSONB | always server-computed, never client-writable |